Пятое задание по программированию

Регрессионные модели в scikit-learn

Краткий обзор подходов, примеры кода и сопоставление с практикой анализа данных

Содержательные акценты
• линейные и полиномиальные модели
• регуляризация, устойчивость и подбор параметров
• сравнение методов и демонстрационные примеры

Далее приведен основной материал по теме с сохранением ключевых примеров, формул и программных фрагментов.

Формат представления: PDF-документ с теоретической частью и практическими иллюстрациями на Python.

<u>Классификаторы Регрессии в scikit learn:</u>

Разбор **регрессионных моделей** из библиотеки scikit-learn, их математическая суть, примеры кода на Python, аналогия с Excel и краткая программа для проверки моделей.

1. Линейные модели (Linear Models)

1.1. Парная и Множественная Линейная Регрессия / МНК

Суть: МНК (Ordinary Least Squares) ищет коэффициенты *w* так, чтобы минимизировать сумму квадратов разниц между фактическими значениями *y* и предсказанными ŷ: $\sum(y_i - \hat{y}_i)^2$.

Когда использовать: Если связь между признаками и целью линейная.

Python (МНК):

```
from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

import numpy as np

# Генерация данных

X = np.array([[1], [2], [3], [4], [5]])

y = np.array([2, 4, 5, 8, 10])

# Модель

model = LinearRegression()

model.fit(X, y)

# Прогноз

print(f"Коэффициент (w1): {model.coef_[0]:.2f}")

print(f"Свободный член (w0): {model.intercept_:.2f}")

print(f"Прогноз для X=6: {model.predict([[6]])[0]:.2f}")
```

Коэффициент (w1): 2.00. Свободный член (w0): -0.20. Прогноз для X=6: 11.80

Excel: Функция =ЛИНЕЙН() или диаграмма "Линия тренда" с галочкой "показывать уравнение".

1.2. Неотрицательный метод наименьших квадратов (NNLS)

Суть: То же, что и МНК, но с ограничением: все веса *w* должны быть ≥0.
Когда использовать: Когда признаки не могут иметь отрицательный вклад в результат (например, интенсивность пикселей, количество товара).

Python:

```
from sklearn.linear_model import LinearRegression

model_nnls = LinearRegression(positive=True) # Параметр positive

model_nnls.fit(X, y)

print(f"NNLS коэф: {model_nnls.coef_}") NNLS коэф: [2.]
```

(В SciKit-Learn 1.0+ это делается через positive=True в LinearRegression, либо отдельный оптимизатор из scipy.optimize.nnls)

1.3. Полиномиальная Регрессия

Суть: Линейная модель, но примененная не к самим признакам xx, а к их степеням [x, x2, x3...]. Позволяет моделировать изгибы. **Когда использовать:** Данные имеют нелинейную форму (парабола, синусоида).

Python:

```
from sklearn.preprocessing import PolynomialFeatures

from sklearn.pipeline import make_pipeline

# Создаем пайплайн: сначала генерируем степени X (x^2), потом применяем линейную регрессию

model_poly = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())

model_poly.fit(X, y)

print(f"Прогноз для X=6 (полином): {model_poly.predict([[6]])[0]:.2f}")
```

Прогноз для X=6 (полином): 12.80

Excel: "Линия тренда" -> "Полиномиальная" (степень 2 или 3).

1.4. Нелинейные регрессии (Экспоненциальная, Логарифмическая, Степенная)

Суть: преобразование данных. Невозможно напрямую обучить "экспоненциальную" модель в линейном алгоритме, но мы можем линеаризовать задачу.

Экспонента $y=ae^{bx}$ берем логарифм от y –> ln(y)=ln(a)+bx Обучаем линейную регрессию на x и ln(y). **Python (Экспонента):**

```
# Экспоненциальная регрессия: y = a * exp(b * x)

log_y = np.log(y) # Логарифмируем целевую переменную

model_exp = LinearRegression()

model_exp.fit(X, log_y)
```

```
# Коэффициенты: b = model_exp.coef_, ln(a) = model_exp.intercept_
b = model_exp.coef_[0]
a = np.exp(model_exp.intercept_)
print(f"Модель: y = {a:.2f} * exp({b:.2f} * x)")
```

Модель: y = 1.55 * exp(0.39 * x)

Excel – "Линия тренда" –> "Экспоненциальная" (автоматически делает логарифмирование за вас).

1.5. Ридж-регрессия (Ridge / L2)

Суть: К штрафу МНК добавляется штраф за величину коэффициентов:

$\sum(y-\hat{y})^2+\alpha\sum w^2$. Это "сжимает" веса, не давая им стать слишком большими.

Когда использовать Мультиколлинеарность (признаки сильно коррелируют) или переобучение.

Python:

```
from sklearn.linear_model import Ridge
model_ridge = Ridge(alpha=1.0) # alpha - сила регуляризации
model_ridge.fit(X, y)
print(f"Ridge коэф: {model_ridge.coef_}")
```

Ridge коэф: [1.81818182]

1.6. Лассо (Lasso / L1)

Суть: Добавляет штраф $\alpha\sum|w|$. В отличие от Ridge, он может **обнулять** веса (отбрасывать признаки), работая как встроенный отбор признаков.

Когда использовать:

Когда предположительно много признаков не важны.

Python:

```
from sklearn.linear_model import Lasso
model_lasso = Lasso(alpha=0.1)
model_lasso.fit(X, y)
print(f"Lasso коэф: {model_lasso.coef_}") # Может быть 0
```

Lasso коэф: [1.95]

1.7. Эластичная сеть (ElasticNet)

Суть: Комбинация L1 и L2 штрафов. Есть два параметра: alpha (общая сила) и l1_ratio (пропорция L1).

Когда использовать: Когда нужно и отбрасывать признаки (как Lasso), но сглаживать их влияние (как Ridge).

Python:

```
from sklearn.linear_model import ElasticNet


model_en = ElasticNet(alpha=0.1, l1_ratio=0.5)
model_en.fit(X, y)
```

```
•••  ▾ ElasticNet
     ElasticNet(alpha=0.1)
```

1.8. Логистическая регрессия (LogisticRegression)

ВАЖНО: Несмотря на название, это **классификатор**, а не регрессор. Предсказывает вероятность принадлежности к классу.

Суть: Линейная комбинация пропускается через сигмоиду, "сжимающую" результат в диапазон [0, 1].

Python

```
from sklearn.linear_model import LogisticRegression
y_binary = np.array([0, 0, 0, 1, 1]) # Классы
clf = LogisticRegression()
clf.fit(X, y_binary)
print(f"Вероятность класса 1 для X=6: {clf.predict_proba([[6]])[0][1]:.2f}")
```

Вероятность класса 1 для X=6: 0.93

1.9. Стохастический градиентный спуск (SGDRegressor)

Суть: Не формула (как МНК), а итеративный метод. Подходит для огромных дан-ных которые не влезают в память.

Когда использовать: Очень большие датасеты (Online learning).

Python:

```
from sklearn.linear_model import SGDRegressor


model_sgd = SGDRegressor(max_iter=1000, tol=1e-3)
model_sgd.fit(X, y.ravel()) # y должен быть одномерным
print(f"SGD коэф: {model_sgd.coef_}")
```

SGD коэф: [1.81670817]

1.10. Квантильная регрессия (QuantileRegressor)

Суть: Предсказывает не среднее значение, а определенный квантиль (например, медиану или 90-й процентиль). Устойчива к выбросам.

Когда использовать: Нужно построить доверительные интервалы или данные с сильными выбросами.

Python:

```
from sklearn.linear_model import QuantileRegressor


# Предсказание медианы (50-й процентиль)
model_quant = QuantileRegressor(quantile=0.5, alpha=0, solver='highs')
model_quant.fit(X, y)
```

```
•••  ▾ QuantileRegressor
     QuantileRegressor(alpha=0)
```

2. Программа для проверки моделей (Шаблон)

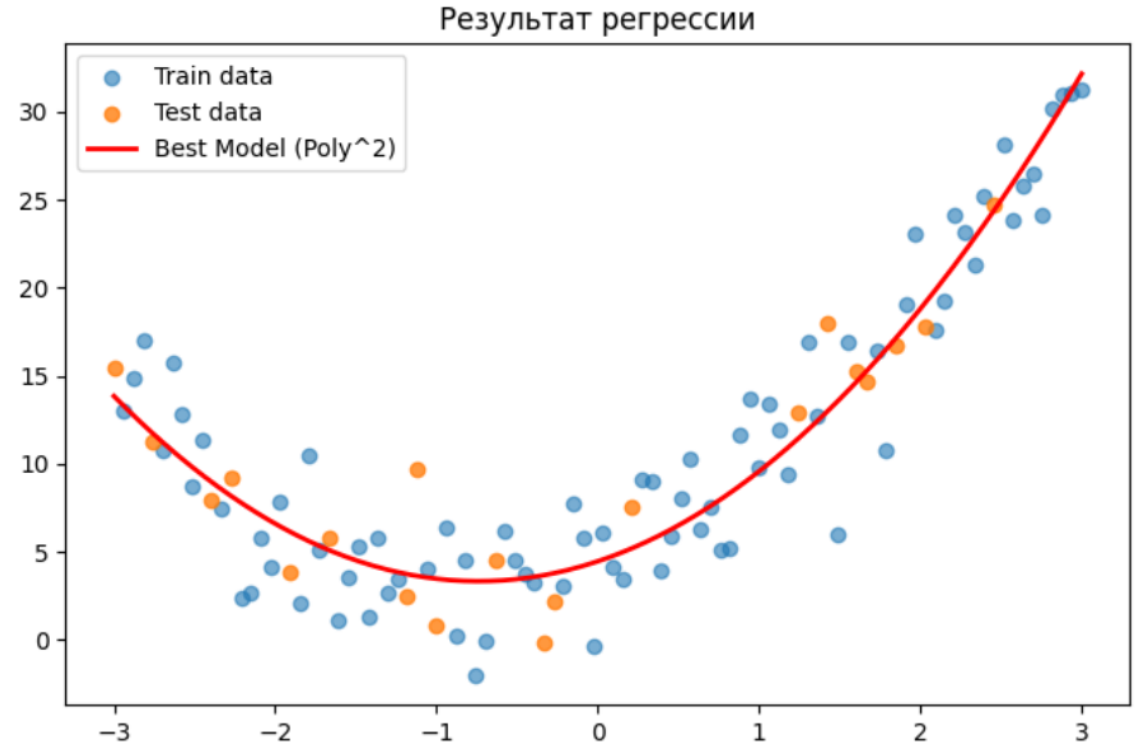
Эта короткая программа позволяет сравнить любые регрессионные модели на синтетических данных.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
```

```
# Список моделей для тестирования
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
# --- 1. Генерация данных (пример: квадратичная зависимость + шум) ---
np.random.seed(42)
X = np.linspace(-3, 3, 100).reshape(-1, 1)
y = 2 * X.ravel()**2 + 3 * X.ravel() + 5 + np.random.normal(0, 3, X.shape[0])
# --- 2. Разделение на обучающую и тестовую выборку ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# --- 3. Словарь моделей для сравнения ---
models = {
    'Linear (Underfit)': LinearRegression(),
    'Ridge (L2)': Ridge(alpha=1.0),
    'Lasso (L1)': Lasso(alpha=0.1),
    'ElasticNet': ElasticNet(alpha=0.1, l1_ratio=0.5),
    'Polynomial (Degree 2)': make_pipeline(PolynomialFeatures(degree=2),
LinearRegression()),
    'Polynomial (Degree 10)': make_pipeline(PolynomialFeatures(degree=10),
Ridge(alpha=0.01)) # С регуляризацией
}
# --- 4. Обучение и оценка ---
print("="*50)
print("Сравнение регрессионных моделей")
print("="*50)
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
```

```
    results[name] = {'MSE': mse, 'R2': r2}
    print(f"{name:25} | MSE: {mse:.3f} | R2: {r2:.3f}")
# --- 5. Визуализация лучшей модели (для примера - полином 2 степени) ---
best_model = models['Polynomial (Degree 2)']
X_plot = np.linspace(-3, 3, 300).reshape(-1, 1)
y_plot = best_model.predict(X_plot)
plt.figure(figsize=(8,5))
plt.scatter(X_train, y_train, alpha=0.6, label='Train data')
plt.scatter(X_test, y_test, alpha=0.8, label='Test data')
plt.plot(X_plot, y_plot, color='red', linewidth=2, label='Best Model (Poly^2)')
plt.legend()
plt.title("Результат регрессии")
plt.show()
```

```
... ==================================================
    Сравнение регрессионных моделей
    ==================================================
    Linear (Underfit)         | MSE: 33.664 | R2: 0.235
    Ridge (L2)                | MSE: 33.575 | R2: 0.237
    Lasso (L1)                | MSE: 33.448 | R2: 0.240
    ElasticNet                | MSE: 33.216 | R2: 0.245
    Polynomial (Degree 2)     | MSE: 5.577 | R2: 0.873
    Polynomial (Degree 10)    | MSE: 7.344 | R2: 0.833
```



3. Задачи для проверки модели (Объяснение)

Когда вы запускаете код выше, вы решаете следующие задачи:

Задача №1: Оценка качества (Ошибки). *Программа:* Выводит MSE и R2.

Объяснение: Смотрим, у какой модели MSE (среднеквадратичная ошибка) *меньше*, а R2 (коэффициент детерминации) *ближе к 1.* Это значит, что модель точнее предсказывает те данные, которые она не видела во время обучения.

Задача №2: Диагностика переобучения

Программа: Сравнивает Polynomial Degree 2 и Polynomial Degree 10.

Объяснение: Если у Degree 10 на тесте R2 очень низкий, а ошибка высокая (хотя на обучении могла быть идеальной), значит модель **переобучилась** (запомнила шум, а не закономерность).

Задача №3: Влияние регуляризации

Программа: Сравнивает Linear, Ridge, Lasso.

Объяснение: Если обычная Линейная регрессия дает плохой результат а Ридж или Лассо — лучше, значит в данных была мультиколлинеарность или переобучение. Регуляризация помогла.

Итог: Каждый метод решает одну и ту же задачу (предсказание числа *y*), но с разными математическими допущениями. Выбор модели - всегда компромисс между простотой (Linear) точностью (Poly) устойчивостью (Ridge/ElasticNet).